

Módulo 1 — Boas-vindas, Ambiente e o Primeiro Programa

Jornada DEV START · Aula 1 · 13/07 (segunda) · Programa START — TOTVS Paulista
Material entregue ao final da Aula 1. Guarde para consultar sempre que precisar.



Onde você está na jornada

[1] → 2 → 3 → 4 → 5 → 6 | 7 → 8 → 9 → 10
▲ Fundamentos (Harbour) | Protheus (ADVPL)
você está aqui

Esta é a **aula inaugural**. Hoje o foco é conhecer a jornada, preparar as ferramentas e — o mais importante — rodar o seu **primeiro programa**. É um dia leve de propósito: a ideia é começar com o pé direito e sem sobrecarga. 🚀



Roteiro da aula (2h)

Bloco	Tempo	O quê
Abertura institucional (gestão TOTVS)	20–30 min	Boas-vindas da empresa e o porquê do programa
Ambientação	~30 min	Conferir o ambiente do pré-curso e resolver pendências
Hello World ao vivo	~40 min	Todos escrevem, compilam e executam o 1º programa
Dúvidas / networking	~10 min	Perguntas abertas e integração da turma

Objetivos do módulo

Ao final desta aula você será capaz de: - Entender o que é o curso e o que vai aprender - Compreender, em linhas gerais, o que são **Harbour** e **ADVPL** - Confirmar seu ambiente de desenvolvimento funcionando - Escrever, compilar e executar seu primeiro programa em Harbour

1.1 Bem-vindo(a) à Jornada

Este curso foi criado para quem quer **aprender a programar do zero** usando **Harbour/ADVPL** — a mesma família de linguagem usada no **ERP Protheus da TOTVS**, um dos sistemas de gestão empresarial mais usados no Brasil.

Você **não precisa** saber nada de programação. Vamos começar pelo começo: como o computador “pensa”, o que é lógica, e como transformar um problema real em código.

A jornada em duas partes

Parte	Aulas	O que você aprende
Fundamentos (Harbour)	1 a 6	Lógica, variáveis, decisões, loops, funções e arrays
Protheus (ADVPL)	7 a 10	Ambiente Protheus, tabelas, CRUD, menus, relatórios, OOP

E no fim: um **projeto real (TCC)** que você constrói e leva no portfólio.

 **Combinado da turma:** programação se aprende **fazendo**. Quanto mais você praticar e perguntar, mais rápido aprende. Ninguém aqui nasceu sabendo.

1.2 O que são Harbour e ADVPL?

Uma breve história (xBase)

- **1980 — dBASE:** primeira linguagem popular para manipular bancos de dados em micros.
- **1985 — Clipper:** evoluiu o dBASE e dominou os sistemas comerciais no Brasil.
- **Anos 2000 — a bifurcação:** com o Windows, o legado Clipper seguiu por dois caminhos:

- **Harbour** → compilador xBase moderno, gratuito e open-source (Windows, Linux, Mac).
- **ADVPL** → o dialeto criado pela **TOTVS** para programar dentro do Protheus.

Por que isso é ótimo para você

Harbour e ADVPL são **mais de 90% compatíveis**. Tudo que você aprender em Harbour nas primeiras aulas vale quase inteiro quando entrarmos no Protheus. **Aprende uma vez, colhe duas.**

Característica	Harbour	ADVPL
Onde roda	Qualquer computador	Dentro do Protheus
Compilador	hbm2 (local)	DevStudio / TDS
Para que serve	Aprendizado, sistemas stand-alone	ERP Protheus
Funções exclusivas	—	xFilial, AxCadastro, mBrowse...

Nas aulas 1–6 usamos **Harbour + VSCode** para aprender os fundamentos. A partir da aula 7, migramos para o **ADVPL** dentro do Protheus.

1.3 Conferindo o ambiente

Você já montou o ambiente no **pré-curso** ([pre-curso/setup-ambiente-harbour.md](#)). Aqui vamos só **confirmar** que está tudo certo. Se você não conseguiu terminar, tudo bem — é agora, no bloco de ambientação, que resolvemos juntos.

Checagem rápida (2 minutos)

Abra o terminal e rode:

```
harbour --version
```

Deve aparecer a versão do Harbour. Se aparecer erro, chame o instrutor no bloco de ambientação.

Você precisa ter: 1. **VSCode** instalado 2. **Harbour** instalado (`harbour --version` funciona) 3. **Extensão “Harbour Language Support”** no VSCode

 Ambiente pronto no VSCode

1.4 Primeiro programa: Hello World

Tradição de toda linguagem: o primeiro programa exibe uma mensagem na tela. Vamos fazer o nosso.

Criando o arquivo

1. Abra a pasta `C:\harbour-curso` no VSCode
2. Crie um arquivo chamado `hello.prg`
3. Digite o código:

```
// Meu primeiro programa em Harbour
// tudo após // é comentário – o compilador ignora

FUNCTION Main()

    QOut("Olá, Mundo!")
    QOut("Bem-vindo(a) à Jornada DEV START!")

RETURN NIL
```

Entendendo cada pedaço

Elemento	O que faz
<code>// comentário</code>	Linha ignorada pelo compilador; serve para documentar
<code>FUNCTION Main()</code>	Define a função principal — todo programa começa por ela
<code>QOut("texto")</code>	Exibe uma linha de texto na tela
<code>RETURN NIL</code>	Encerra a função sem devolver nada (<code>NIL</code> = “nada”)

Compilando e executando

No terminal do VSCode:

```
hbm2 hello.prg
hello.exe
```

Saída esperada:

Olá, Mundo!
Bem-vindo(a) à Jornada DEV START!

O que é **hbmK2** ? É o “montador” do Harbour: pega seu arquivo **.prg** e gera um executável **.exe** . Escrever → compilar → executar é o ciclo que você vai repetir o tempo todo.

💡 **Atalho pra testar rápido:** o Harbour também tem o **hbrun** , um interpretador/runner de scripts. **hbrun hello.prg** executa o programa **na hora, sem gerar o .exe** — ótimo pra experimentar. Neste curso usamos o **hbmK2** como padrão, porque entender a **compilação** vai valer ouro quando chegarmos no **Protheus/ADVPL**.

Brincando com a saída

```
FUNCTION Main()

    QOut("Linha 1")           // QOut pula linha ao final
    QOut("Linha 2")

    QQOut("Sem ")             // QQOut NÃO pula linha
    QQOut("pular linha")

    ? "Atalho para QOut"      // ? é atalho de QOut
    ?? " continua na mesma linha"  // ?? é atalho de QQOut

RETURN NIL
```

1.5 A estrutura de um programa Harbour

```
// =====  
// NOME: exemplo.prg  
// DESCRIÇÃO: estrutura básica  
// =====  
  
FUNCTION Main()  
  
    LOCAL cNome := "Maria"    // variável de texto  
    LOCAL nIdade := 28        // variável numérica  
  
    QQout("Nome: " + cNome)  
    QQout("Idade: " + Str(nIdade))  
  
RETURN NIL  
  
// outras funções vêm depois da principal  
FUNCTION OutraFuncao()  
    // ...  
RETURN NIL
```

Regras básicas para guardar

- Arquivos Harbour têm extensão **.prg**
- O programa **começa pela função Main()**
- **Não** é case-sensitive: **FUNCTION** = **function** = **Function**
- Comentários: **//** (uma linha) ou **/* ... */** (bloco)

Mão na massa (em aula)

Vamos fazer juntos, ao vivo:

Crie um arquivo **apresentacao.prg** que exiba uma “ficha” formatada assim:

```
=====  
FICHA DE APRESENTAÇÃO  
=====  
Nome      : [seu nome]  
Cidade    : [sua cidade]  
Curso     : Harbour/ADVPL  
=====
```

Faremos passo a passo — e no fim cada um roda a sua ficha. 🎯

1.6 Guardando seu código: controle de versão com Git e GitHub

Todo o código que você escrever no curso será **entregue pelo GitHub** — e isso não é burocracia: é como profissionais trabalham. Vale entender o essencial.

Por que controle de versão?

Pense em salvar um trabalho como `relatorio_v2_final_AGORA_VAI.doc`. Controle de versão resolve isso de forma profissional, dando a você: - **Histórico**: um “Ctrl+Z infinito” — volte a qualquer versão anterior do projeto. - **Backup na nuvem**: seu código não se perde se o computador falhar. - **Portfólio**: seu trabalho fica visível e organizado para mostrar a recrutadores.

🧠 É como **salvar o jogo** a cada fase: se algo der errado, você volta ao último save.

Git x GitHub

- **Git** é a ferramenta que registra a **linha do tempo** do seu projeto — roda no **seu computador**.
- **GitHub** é o serviço onde seu **repositório** (a pasta do projeto versionada) fica **na nuvem**.
- Em resumo: você **versiona** com o Git (local) e **publica/entrega** no GitHub (nuvem).

O ciclo do dia a dia (3 passos)

Passo	Comando	O que faz
1. Selecionar	<code>git add .</code>	marca os arquivos que entram na próxima “foto”
2. Salvar	<code>git commit -m "mensagem"</code>	tira a “foto” do projeto, com um rótulo
3. Enviar	<code>git push</code>	manda os commits para o GitHub (nuvem)

Depois do `push`, copie o **link do repositório** e cole na atividade do Classroom.

 O passo a passo completo — criar conta, instalar o Git e o primeiro **push** (com prints) — está no guia **pre-curso/setup-github.md**. Aqui foi só o mapa mental.

Referência rápida do Módulo 1

```
FUNCTION Main() ... RETURN NIL // toda aplicação começa aqui
QOut("texto")                 // imprime e pula linha (atalho: ? )
QQOut("texto")                 // imprime sem pular linha (atalho: ?? )
Str(nNumero)                   // converte número em texto
"a" + "b"                      // concatena (junta) textos
// comentário de linha        /* comentário de bloco */
```

Compilar e rodar:

```
hbm2 arquivo.prg
arquivo.exe
```



Exercícios

Os exercícios desta aula estão em **exercicios.md** (nesta mesma pasta). **Prazo sugerido:** até a Aula 2 (14/07).

Para a próxima aula

Módulo 2 — Lógica, algoritmos e fluxogramas. Antes de escrever mais código, vamos aprender a **pensar como programador**: o que é um algoritmo, como desenhar a solução no papel e por que isso economiza horas depois.



Antes da Aula 2: garanta que seu Hello World rodou. Se travou em algo, poste no Discord — a gente destrava junto.